Objects & Arrays

CRUD on Objects & Arrays — In-Class Discovery Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Keep your **browser** and **DevTools Console** open beside you. Write today's code in a JavaScript file linked to your HTML page, and **refresh the page after every step** to see the output. **Attempt and write each answer *before* we discuss it** — your first guess matters more than being right. As in Class 19, declare everything with `const`. You already know **Python**, so we'll keep comparing JavaScript to Python as we go.

Part 1 — Many Details, One Student: Objects

TRY THIS · SCRIPT.JS

In `script.js`, create **four separate variables** to store one student's details: their **name**, **city**, **college**, and **roll number**. Use meaningful **camelCase** names (Class 19). Then log all four in a single `console.log()`.

Q1. All four variables describe the *same* student — but does anything in your code actually **tie them together**? If someone else read your code, how would they know these four values belong to one person?

Q2. Now imagine your college has **500 students**. How many variables would you need? What would you name the ones for student 2, student 3, … student 500? And what happens the day you decide every student also needs a **phone number**?

Concept — Separate Variables Don't Scale. One variable per detail works for one student, but it quickly becomes unmanageable: the number of variables **multiplies** with every student and every new detail, the names get long and confusing, and **nothing in the code groups the values that belong together**. We need a way to keep all of one student's details together, under a **single name**.

Concept — Objects. In JavaScript, an **object** is a collection of related data stored as **key-value pairs**, all under one variable name. Each **key** is a label for its **value**, and the value can be any datatype — a string, a number, a boolean, even another object. A key-value pair is also called a **property**. This is the one **non-primitive** datatype from Class 19's list of 8.
Syntax: wrap the properties in curly braces `{ }`, write each one as `key: value`, and separate them with commas.

TRY THIS · SCRIPT.JS

Comment out your four variables, then write:

```
const student = {
  name: "Likhilesh",
  city: "Delhi",
  college: "KRMU",
  rollNumber: "2616400901"
};
console.log(student);
console.log(typeof student);
```

Q3. What did `console.log(student)` show in the console? Click the small **arrow** next to it to expand it — what do you see? What did `typeof` print?

Q4. Compare this with your four separate variables. How many **variable names** do you need now? If you wanted to store a **phone number** for this student, what would you change — and how is that easier than before?

Q5. In your own words: what is an **object** in JavaScript, and when would you reach for one instead of separate variables?

Concept — Python Analogy: Dictionary vs Object. If this looks familiar, it should — a JavaScript object looks a lot like a **Python dictionary**.

Python:

```
student = {
    "name": "Likhilesh",
    "city": "Delhi",
    "college": "KRMU",
    "roll_number": "2616400901"
}
```

JavaScript:

```
const student = {
  name: "Likhilesh",
  city: "Delhi",
  college: "KRMU",
  rollNumber: "2616400901"
};
```

	Python dictionary	**JavaScript object**
Wrapped in	Curly braces `{ }`	Curly braces `{ }`
Each entry	`key: value`	`key: value`
Separated by	Commas	Commas
Values	Any type	Any type
Keys	Must be written in quotes (`"name"`)	Usually written without quotes (`name`); quotes are optional
Key types	Strings, numbers, tuples, …	JavaScript converts every key to a string
Called	Key-value pairs / items	Key-value pairs / properties
Naming style	snake_case (`roll_number`)	camelCase (`rollNumber`)

Q6. Looking at the Python and JavaScript versions side by side, write **two things that are the same** and **two things that are different**.

Part 2 — CRUD on Objects

Concept — CRUD. Almost everything a program does with stored data comes down to four operations: **Create** (make something new), **Read** (look something up), **Update** (change it), and **Delete** (remove it). Together they're called **CRUD**. You already did your first Create in Part 1, when you wrote the `student` object.

Note: Keep adding the code below under your `student` object from Part 1, in the same file. Every step changes `student`, so read the **one-line preview** that `console.log()` prints for each step. If you expand an older log with the arrow, remember:
Chrome shows you the object as it is now, not as it was when it was logged.

TRY THIS · SCRIPT.JS

```
console.log(student.name);
console.log(student["city"]);
```

Q7. What did each line print? Both **read** a value from the object — one uses a dot (**dot notation**), the other uses square brackets with the key in quotes (**bracket notation**). Which of the two would also work on a **Python dictionary**?

TRY THIS · SCRIPT.JS

```
console.log(student.phone);
```

Q8. `student` has no `phone` key. What did JavaScript give you? What would **Python** do if you asked a dictionary for a key that doesn't exist?

TRY THIS · SCRIPT.JS

```
student["country"] = "INDIA";
console.log(student);

student["city"] = "Mumbai";
console.log(student);

student.college = "Zenith School of AI";
console.log(student);
```

Q9. The `country` line and the `city` line use exactly the same syntax — but one **added** something new and the other **replaced** something. Which did which, and how does JavaScript decide whether to add or replace? Is it same in python?

TRY THIS · CONSOLE

In the **Console** (not in your file), type:

```
student = { name: "Someone Else" };
```

Q10. Class 19 said `const` signals "this value is locked in" — yet you just added and changed properties on a `const` object without any error. What happened when you tried to **replace** `student` entirely?

TRY THIS · SCRIPT.JS

```
delete student["name"];
console.log(student);

delete student.city;
console.log(student);
```

Q11. What did `delete` do to the object each time? Did it work with both bracket and dot notation? How would you remove a key from a **Python** dictionary?

TRY THIS · SCRIPT.JS

```
const student2 = student;
student2.city = "Jaipur";

console.log(student);
console.log(student2);
```

Q12. You only changed `student2` — so why does `student` show `city: "Jaipur"` too? Did `const student2 = student` create a **second object**, or just a **second name** for the same object?

Concept — Copy by Reference. Assigning an object to another variable does **not** copy it. Both names now point to the **same object**, so a change made through one name shows up through the other. Python behaves the same way: `student2 = student` gives you two names for one dictionary. Making a real, separate copy needs a different technique, which we'll see later.

Q13. Write one line of code for each **CRUD** operation on an object `car` with the keys `brand`, `model`, and `year`: **Create** it, **Read** its model, **Update** its year, and **Delete** its brand.

Part 3 — CRUD on Arrays

Concept — Arrays. An object is perfect for different facts about *one* thing. But what about a **list of similar things** — like a list of fruits? For that, JavaScript has the **array**: an **ordered list of values** stored under one name, written inside square brackets `[ ]` and separated by commas. If you know Python, this is JavaScript's version of a **list**.

TRY THIS · SCRIPT.JS

```
const fruitsArr = ["Mango", "Grapes"];
console.log(fruitsArr);

const fruitsObj = {
  0: "Mango",
  1: "Grapes"
};
console.log(fruitsObj);

console.log(typeof fruitsArr);
console.log(typeof fruitsObj);
```

Q14. Compare the two logs. In `fruitsObj`, the keys are `0` and `1` — what do the 0 and 1 seem to be in `fruitsArr`? And what did `typeof` say for each — what does that tell you about what an array **really is**?

Concept — An Array Is a Special Object. An array is an object whose keys are **positions**, called **indexes**, numbered automatically from **0**. You don't write the keys yourself — JavaScript keeps them in order for you. That's why `typeof` shows `"object"` for both.

TRY THIS · SCRIPT.JS

```
console.log(fruitsArr[0]);
console.log(fruitsArr[1]);
console.log(fruitsArr.length);
console.log(fruitsArr[fruitsArr.length - 1]);
```

Q15. What did each line print? If an array has `length` items, why is the index of the **last** item always `length - 1`?

TRY THIS · SCRIPT.JS

```
console.log(fruitsArr[-1]);
```

Q16. What did JavaScript print? What would `my_list[-1]` give you in **Python**?

TRY THIS · SCRIPT.JS

```
fruitsArr[2] = "Apple";
console.log(fruitsArr);
```

Q17. Did adding `"Apple"` at index 2 work? What did you need to **know** about the array to pick the right index — and what could go wrong if you picked the wrong one?

TRY THIS · SCRIPT.JS

```
fruitsArr[20] = "Watermelon";
console.log(fruitsArr);
console.log(fruitsArr.length);
```

Q18. What does the console show between `"Apple"` and `"Watermelon"`? What is the **length** now — does it match the number of fruits you actually added? What would **Python** do if you assigned to index 20 of a 3-item list?

Note: Now **comment out** the `fruitsArr[20]` line and refresh, so the array goes back to 3 fruits before you continue.

Concept — Holes. Assigning to an index past the end leaves **empty slots** — "holes" — in the array, and its length no longer matches the real number of items. That's not something you ever want. So to add or remove items, we use **array methods**: built-in commands that keep the array in order for you.

TRY THIS · SCRIPT.JS

```
fruitsArr.unshift("Watermelon");
fruitsArr.push("Banana");
console.log(fruitsArr);
```

Q19. Where did `"Watermelon"` go, and where did `"Banana"` go? Which **Python** list method does the same job as `push()`? Which one does what `unshift()` did?

Hint: There's another method, `splice()`, that can also **add** items — we'll come back to it at the end of this part.

TRY THIS · SCRIPT.JS

```
fruitsArr.shift();
fruitsArr.pop();
console.log(fruitsArr);
```

Q20. Which fruit did `shift()` remove, and which did `pop()` remove? How would you remove the first and the last item of a **Python** list?

Hint: `splice()` can **remove** items too — hold that thought until the end of this part.

TRY THIS · SCRIPT.JS

```
delete fruitsArr[1];
console.log(fruitsArr);
console.log(fruitsArr.length);
```

Q21. You used `delete` on objects in Part 2. What did it leave behind at index 1 of the array? Did the length change? Why is this a **bad way** to remove an item from an array?

Note: Comment out the `delete` line and refresh before you continue.

TRY THIS · SCRIPT.JS

```
fruitsArr[1] = "Orange";
console.log(fruitsArr);
```

Q22. Q17 and Q18 showed that using `[ ]` to *add* items is risky. Why is `fruitsArr[1] = "Orange"` perfectly **safe**?

Hint: Yes — `splice()` can do this too. Time to see how.

TRY THIS · SCRIPT.JS

```
fruitsArr.push("Grapes", "Watermelon");
console.log(fruitsArr);

fruitsArr.splice(2, 1);
console.log(fruitsArr);

fruitsArr.splice(1, 2);
console.log(fruitsArr);
```

Q23. Which fruit did `splice(2, 1)` remove? Which fruits did `splice(1, 2)` remove? Unlike `pop()` and `shift()`, `splice()` can remove from **anywhere** — what do you think the **first number** and the **second number** each mean?

TRY THIS · SCRIPT.JS

```
fruitsArr.splice(1, 0, "Kiwi");
console.log(fruitsArr);

fruitsArr.splice(2, 0, "Guava", "Papaya");
console.log(fruitsArr);
```

Q24. This time the second number is `0`. Did anything get removed? What happened to the **extra values** you passed after the 0?

TRY THIS · SCRIPT.JS

```
fruitsArr.splice(0, 1, "Pineapple");
console.log(fruitsArr);
```

Q25. What happened to `"Mango"`? In one call, `splice()` removed something *and* added something — which **CRUD** operation does that amount to?

Concept — splice(): One Method for Create, Update and Delete. `splice(start, deleteCount, item1, item2, ...)` goes to index `start`, removes `deleteCount` items there, then inserts any items you pass after that.
Delete: `splice(2, 1)` — remove 1 item at index 2
Create: `splice(2, 0, "Kiwi")` — remove nothing, insert at index 2
Update: `splice(2, 1, "Kiwi")` — remove 1 item and put a new one in its place

Concept — Python Analogy: List Methods.

JavaScript	Python	What it does
`arr.length`	`len(lst)`	Number of items
`arr[arr.length - 1]`	`lst[-1]`	Last item
`arr.push(x)`	`lst.append(x)`	Add to the end
`arr.unshift(x)`	`lst.insert(0, x)`	Add to the start
`arr.pop()`	`lst.pop()`	Remove the last item
`arr.shift()`	`lst.pop(0)`	Remove the first item
`arr.splice(i, 0, x)`	`lst.insert(i, x)`	Insert at index i
`arr.splice(i, 1)`	`del lst[i]`	Remove the item at index i

TRY THIS · SCRIPT.JS

```
const fruitsArr2 = fruitsArr;
fruitsArr2.push("Cherry");

console.log(fruitsArr);
console.log(fruitsArr2);
```

Q26. You only pushed into `fruitsArr2` — did `"Cherry"` show up in `fruitsArr` too? Where have you seen this behavior **before** in today's worksheet?

READ AHEAD · BEFORE THE NEXT SESSIONS

Shallow Copy vs Deep Copy. Today, `const student2 = student` and `const fruitsArr2 = fruitsArr` didn't copy anything. They gave you a *second name* for the same object or array (copy by reference). There are two ways to make a real, separate copy, and we'll discuss them in an upcoming session. Before then, read about:

- **Shallow copy**: what gets copied, and what is still *shared* when an object holds another object or array inside it?
- **Deep copy**: how is it different from a shallow copy?

Terms to search for: the **spread operator** (`{ ...student }` and `[ ...fruitsArr ]`) and `structuredClone()`. In Python, compare `copy.copy()` and `copy.deepcopy()`.